

Wirtualne środowiska obliczeniowe

Mateusz Chmurzyński, Filip Kiernożek

Czerwiec 2026

1 Temat projektu

Realizacja projektu związana jest z automatyzacją zarządzania maszynami wirtualnymi. Składa się na to napisanie zestawu skryptów lub programu ułatwiającego zarządzanie maszynami wirtualnymi lub klastrem, opracowanie odpowiednich scenariuszy i samo użycie narzędzi. Temat projektu (nr 3) zakłada zapoznanie się z tematem wysokiej dostępności. Jest to m. in. wykrywanie awarii (np. protokół heartbeat), przełączanie, dla serwisów stanowych, synchronizacja dysków (np. drbd, nfs). W ramach tematu należy sprawdzić lub skomentować: czas i skutek przełączenia, ograniczenia użytych technologii.

2 Konfiguracja środowisk

2.1 Maszyny wirtualne

W realizacji projektu skonfigurowano dwa laptopy jako węzły klastra:

- VM1 - maszyna wirtualna 1, uruchomiona na maszynie gospodarza Filipa z systemem operacyjnym gospodarza macOS,
- VM2 - maszyna wirtualna 2, uruchomiona na maszynie gospodarza Mateusza z systemem operacyjnym gospodarza Windows.

W tabeli 1 przedstawiono parametry obu maszyn wirtualnych

Parametr	Wartość
System operacyjny	Ubuntu Server 26.04 LTS
Liczba maszyn	2
Liczba rdzeni CPU	2
Pamięć RAM	2 GB
Dysk systemowy A	20 GB
Dysk systemowy B	5 GB
Dysk DRBD (partycja B)	500 MB

Tabela 1: Parametry maszyn wirtualnych wykorzystanych w projekcie

```

filip@vm1:~$ hostnamectl --static && uname -m && lsblk
vm1
aarch64
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
vda                                  253:0    0   20G  0 disk
├─vda1                              253:1    0   953M  0 part /boot/efi
├─vda2                              253:2    0    1.8G  0 part /boot
└─vda3                              253:3    0   17.3G  0 part
   └─ubuntu--vg-ubuntu--lv          252:0    0    10G  0 lvm  /
vdb                                  253:16   0     5G  0 disk
└─vdb1                              253:17   0   500M  0 part
   └─drbd0                          147:0    0  499.9M  0 disk /data

```

Rysunek 1: VM1 - szczegóły

```

filip@vm2:~$ hostnamectl --static && uname -m && lsblk
vm2
x86_64
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0     0   20G  0 disk
├─sda1                              8:1     0   953M  0 part /boot/efi
├─sda2                              8:2     0    1.8G  0 part /boot
└─sda3                              8:3     0   17.3G  0 part
   └─ubuntu--vg-ubuntu--lv          252:0    0    10G  0 lvm  /
sdb                                  8:16    0     5G  0 disk
└─sdb1                              8:17    0   500M  0 part
   └─drbd0                          147:0    0  499.9M  0 disk
sr0                                 11:0    1  1024M  0 rom

```

Rysunek 2: VM2 - szczegóły

2.2 Sieć

Maszyny były uruchomione w trybie sieci mostkowej (bridged), dzięki czemu otrzymywały adresy IP z lokalnej sieci gospodarza.

```

filip@vm1:~$ ip -4 -br addr show
lo                UNKNOWN      127.0.0.1/8
enp0s1            UP           192.168.0.122/24 metric 100
wg0               UNKNOWN      10.0.0.1/24  10.0.0.100/24
tailscale0        UNKNOWN      100.100.169.89/32

```

Rysunek 3: VM1 - sieci

```

filip@vm2:~$ ip -4 -br addr show
lo                UNKNOWN      127.0.0.1/8
enp0s3            UP           192.168.0.239/24 metric 1024
wg0               UNKNOWN      10.0.0.2/24
tailscale0        UNKNOWN      100.120.67.77/32

```

Rysunek 4: VM2 - sieci

Ze względu na wykorzystanie dwóch niezależnych sieci domowych oraz obecność CGNAT zastosowano Tailscale jako warstwę zapewniającą wzajemną osiągalność węzłów przez Internet. Nie zostało to przewidziane na etapie koncepcji.

```

filip@vm1:~$ sudo tailscale status
100.100.169.89  vm1  kiernozek16@  linux  offline

100.120.67.77  vm2  kiernozek16@  linux  active; direct 109.241.129.170:41641; offline, last seen 11m ago, tx 736155328
92 rx 582976884

# Health check:
#   - Unable to connect to the Tailscale coordination server to synchronize the state of your tailnet. Peer reachability might degrade over time.

```

Rysunek 5: VM1 - Tailscale

```

filip@vm2:~$ sudo tailscale status
100.120.67.77  vm2  kiernozek16@  linux  -

100.100.169.89  vm1  kiernozek16@  linux  active; direct 217.119.75.214:43851; offline, last seen 2m ago, tx 648374584
rx 11326016

```

Rysunek 6: VM2 - Tailscale

Właściwa komunikacja klastra realizowana była przez tunel WireGuard. Tailscale zapewniał jedynie możliwość zestawienia połączenia pomiędzy hostami znajdującymi się za translacją adresów.

```

filip@vm1:~$ sudo wg show
interface: wg0
  public key: 56PMaq5gdPhQWdEfdNs0r0lsRh39bK8RQcwQFC+NZUs=
  private key: (hidden)
  listening port: 51820

peer: saBOU1hFEoB+d01IRMHXmDDqbbzL9R1pnaC0v2RC4Bc=
  endpoint: 100.120.67.77:58013
  allowed ips: 10.0.0.0/24
  latest handshake: 50 seconds ago
  transfer: 761.96 MiB received, 134.34 GiB sent
  persistent keepalive: every 25 seconds

```

Rysunek 7: VM1 - WireGuard

```

filip@vm2:~$ sudo wg show
interface: wg0
  public key: saBOU1hFEoB+d01IRMHXmDDqbbzL9R1pnaC0v2RC4Bc=
  private key: (hidden)
  listening port: 58013

peer: 56PMaq5gdPhQWdEfdNs0r0lsRh39bK8RQcwQFC+NZUs=
  endpoint: 100.100.169.89:51820
  allowed ips: 10.0.0.0/24
  latest handshake: 45 seconds ago
  transfer: 9.54 MiB received, 671.74 MiB sent
  persistent keepalive: every 25 seconds

```

Rysunek 8: VM2 - WireGuard

2.3 Kłaster wysokiej dostępności

Corosync odpowiadał za wymianę heartbeat pomiędzy węzłami, natomiast Pacemaker zarządzał zasobami klastra i realizował mechanizm failover.

```

[filip@vm1:~$ sudo pcs status
Cluster name: ha_cluster
Cluster Summary:
 * Stack: corosync (Pacemaker is running)
 * Current DC: vm2 (version 3.0.1-3.0.1) - partition with quorum
 * Last updated: Tue Jun  9 21:39:08 2026 on vm1
 * Last change: Wed Jun 10 20:43:23 2026 by hacluster via hacluster on vm1
 * 2 nodes configured
 * 5 resource instances configured

Node List:
 * Online: [ vm1 vm2 ]

Full List of Resources:
 * Clone Set: drbd_r0-clone [drbd_r0] (promotable):
   * Promoted: [ vm1 ]
   * Unpromoted: [ vm2 ]
 * drbd_fs      (ocf:heartbeat:Filesystem):      Started vm1
 * vip (ocf:heartbeat:IPaddr2):                Started vm1
 * fileapp      (systemd:fileapp):              Started vm1

Daemon Status:
 corosync: active/enabled
 pacemaker: active/enabled
 pcsd: active/enabled

```

Rysunek 9: VM1 - klaster HA

```

[filip@vm2:~$ sudo pcs status
Cluster name: ha_cluster
Cluster Summary:
 * Stack: corosync (Pacemaker is running)
 * Current DC: vm2 (version 3.0.1-3.0.1) - partition with quorum
 * Last updated: Wed Jun 10 20:50:40 2026 on vm2
 * Last change: Wed Jun 10 20:43:23 2026 by hacluster via hacluster on vm1
 * 2 nodes configured
 * 5 resource instances configured

Node List:
 * Online: [ vm1 vm2 ]

Full List of Resources:
 * Clone Set: drbd_r0-clone [drbd_r0] (promotable):
   * Promoted: [ vm1 ]
   * Unpromoted: [ vm2 ]
 * drbd_fs      (ocf:heartbeat:Filesystem):      Started vm1
 * vip (ocf:heartbeat:IPaddr2):                Started vm1
 * fileapp      (systemd:fileapp):              Started vm1

Daemon Status:
 corosync: active/enabled
 pacemaker: active/enabled
 pcsd: active/enabled

```

Rysunek 10: VM2 - klaster HA

DRBD realizował synchroniczną replikację danych pomiędzy węzłami w trybie Protocol C. Zapis był uznawany za zakończony dopiero po potwierdzeniu przez oba węzły.

```

filip@vm1:~$ sudo drbdadm status r0
r0 role:Primary
disk:UpToDate
peer connection:Connecting

```

Rysunek 11: VM1 - DRBD

```

filip@vm2:~$ sudo drbdadm status r0
r0 role:Primary
disk:UpToDate

```

Rysunek 12: VM2 - DRBD

System plików wykorzystywany przez aplikację był umieszczony na urządzeniu DRBD i montowany pod katalogiem /data.

```

filip@vm1:~$ df -h /data && ls -la /data/uploads/
Filesystem      Size  Used Avail Use% Mounted on
/dev/drbd0      452M   60M  358M  15% /data
total 60424
drwxr-xr-x  2 root root    4096 Jun  9 21:22 .
drwxr-xr-x  4 root root    4096 Jun  9 20:52 ..
-rw-r----- 1 root root 52428800 Jun  9 21:22 .test5-20260609-223434.bin.e4ifgsvl.part
-rw-r----- 1 root root 1048576 Jun  9 21:04 delta-1.bin
-rw-r----- 1 root root 1048576 Jun  9 21:04 delta-2.bin
-rw-r----- 1 root root 1048576 Jun  9 21:04 delta-3.bin
-rw-r----- 1 root root 1048576 Jun  9 21:11 partition-test.bin
-rw-r----- 1 root root 5242880 Jun  9 20:54 test2b-20260609-221140.bin

```

Rysunek 13: VM1 - pliki

```

filip@vm2:~$ sudo df -h /data && ls -la /data/uploads/
Filesystem      Size  Used Avail Use% Mounted on
/dev/drbd0      452M   60M  358M  15% /data
total 60424
drwxr-xr-x  2 root root    4096 Jun  9 21:22 .
drwxr-xr-x  4 root root    4096 Jun  9 20:52 ..
-rw-r----- 1 root root 52428800 Jun  9 21:22 .test5-20260609-223434.bin.e4ifgsvl.part
-rw-r----- 1 root root 1048576 Jun  9 21:04 delta-1.bin
-rw-r----- 1 root root 1048576 Jun  9 21:04 delta-2.bin
-rw-r----- 1 root root 1048576 Jun  9 21:04 delta-3.bin
-rw-r----- 1 root root 1048576 Jun  9 21:11 partition-test.bin
-rw-r----- 1 root root 5242880 Jun  9 20:54 test2b-20260609-221140.bin

```

Rysunek 14: VM2 - pliki

2.4 Aplikacja testowa HTTP fileapp

Na klastrze uruchomiono prostą aplikację napisaną w Pythonie umożliwiającą wysyłanie i pobieranie plików. Dostęp do aplikacji odbywał się wyłącznie przez wirtualny adres IP klastra (VIP).

```

filip@vm1:~$ curl -s http://10.0.0.100:8080/health && echo && curl -s http://10.0.0.100:8080/
{"ok": true, "node": "vm1", "mounted": true}
{"files": [".test5-20260609-223434.bin.e4ifgsvl.part", "delta-1.bin", "delta-2.bin", "delta-3.bin", "partition-test.bin", "test2b-20260609-221140.bin"], "node": "vm1"}
filip@vm1:~$

```

Rysunek 15: VM1 - serwis HTTP

```

filip@vm2:~$ curl -s http://10.0.0.100:8080/health && echo && curl -s http://10.0.0.100:8080/
{"ok": true, "node": "vm1", "mounted": true}
{"files": [".test5-20260609-223434.bin.e4ifgsvl.part", "delta-1.bin", "delta-2.bin", "delta-3.bin", "partition-test.bin", "test2b-20260609-221140.bin"], "node": "vm1"}
filip@vm2:~$

```

Rysunek 16: VM2 - serwis HTTP

3 Testowanie klastra

Na skonfigurowanym klastrze przeprowadzono serię testów opisanych w kolejnych podsekcjach. Znajdują się tam również wyniki i wnioski.

3.1 Test 1 - awaria aktywnego węzła

Maszyna VM2 działa jako aktywna, a klient wysyła zapytania na adres VIP. Następnie VM2 zostaje przeniesiona w stan **standby** - planowane wyłączenie zasobów. Celem testu jest sprawdzenie, po jakim czasie VM1 przejmie usługę.

W tym teście mierzymy czas od awarii do pierwszej poprawnej odpowiedzi z VM1, liczbę błędnych żądań oraz to, czy klient nadal korzysta z tego samego adresu IP.

Pacemaker wykrył zmianę stanu i rozpoczął migrację zasobów na drugi węzeł. Statystyki przedstawiamy poniżej.

Metryka	Wynik
Czas niedostępności usługi	9.85 s
Liczba błędnych żądań klienta	5
Zmiana adresu IP klienta	Nie
Aktywny węzeł przed przełączeniem	VM2
Aktywny węzeł po przełączeniu	VM1
Adres VIP przed i po przełączeniu	10.0.0.100

Tabela 2: Wyniki testu 1 - planowane przełączenie zasobów

Użytkownik serwisu w poszczególnych chwilach czasowych obserwował zdarzenia opisane w tabeli 3.

Czas [s]	Obserwacja
9.63	Ostatnia poprawna odpowiedź HTTP 200 z VM2.
10.24	Błąd <code>connection refused</code> .
10.81	Błąd <code>connection refused</code> .
11.38	Błąd <code>timeout</code> .
14.08	Błąd <code>timeout</code> .
16.79	Błąd <code>timeout</code> .
19.48	Pierwsza poprawna odpowiedź HTTP 200 z VM1.

Tabela 3: Obserwacje klienta podczas przełączania zasobów

Usługa była niedostępna ok. 10 s. Wpływ na taki czas na pewno miała przepustowość sieci - komputery były umieszczone w dwóch lokalizacjach. Ciekawostka - przeprowadzając test lokalnie (dwie maszyny obok siebie w tej samej sieci lokalnej) ten czas wynosił ok. 3 s.

3.2 Test 2 - zachowanie danych po failover

Przed awarią wysyłany jest wygenerowany plik (5 MiB) do aplikacji `fileapp`. Następnie wyłączana jest VM1, wykonywane jest przełączenie na VM2, a potem pobierany jest ten sam plik z adresu VIP. Poprawność danych zostanie sprawdzona przez porównanie sum kontrolnych. Test zakończył się sukcesem, ponieważ uzyskano te same sumy kontrolne pliku przy zapisie i pobraniu oraz dodatkowo sprawdzono pliki bajt po bajcie. Wyniki zestawiono w tabelach.

Parametr	Wynik
Rozmiar pliku testowego	5 242 880 B (5 MiB)
Adres używany przez klienta	10.0.0.100
Aktywny węzeł przed przełączeniem	VM1
Aktywny węzeł po przełączeniu	VM2
Zgodność sum SHA-256	Tak
Zgodność bajt-po-bajcie (<code>cmp</code>)	Tak
Utrata danych	Nie
Modyfikacja danych	Nie

Tabela 4: Wyniki testu 2 - zachowanie danych po failoverze

Przedstawiamy tabelę ze skrótami. Test został zakończony pomyślnie.

Etap	Skrót SHA-256	Wynik
Plik referencyjny	9234d3c4b70021ab6c1b4d8c4509cb5ba1fd66110560feb5afde939014539b13	Referencja
Przed failoverem	9234d3c4b70021ab6c1b4d8c4509cb5ba1fd66110560feb5afde939014539b13	Zgodne
Po failoverze	9234d3c4b70021ab6c1b4d8c4509cb5ba1fd66110560feb5afde939014539b13	Zgodne

Tabela 5: Weryfikacja integralności danych po failoverze

3.3 Test 3 - powrót uszkodzonego węzła

Zgodnie z koncepcją, po awarii węzła aktywnego, drugi węzeł przejmuje usługę, a następnie uszkodzony węzeł jest ponownie uruchamiany. Sprawdzane będzie, czy DRBD wykonuje resynchronizację danych oraz czy usługa pozostaje dostępna w trakcie tego procesu.

W naszym scenariuszu testowym, po wcześniejszych operacjach, węzłem aktywnym była VM1, dlatego symulacji zatrzymania i ponownego uruchomienia poddano VM2. Logika testu pozostaje identyczna – testujemy mechanizm klastra, nie konkretne etykiety węzłów.

Metryka	Wartość
Liczba żądań HTTP	120
Żądania udane (HTTP 200)	120 (100%)
Żądania błędne	0
Węzeł obsługujący usługę	VM1 przez cały czas
Średnia / maksymalna latencja	1.3 ms / 11 ms
Stan DRBD przed zatrzymaniem VM2	Connected, UpToDate/UpToDate
Stan DRBD po zatrzymaniu VM2	StandAlone, UpToDate/DUnknown
Wielkość synchronizacji przyrostowej	3248 sektorów = 1.66 MiB
Czas resynchronizacji	< 15 s
Stan końcowy DRBD	Connected, UpToDate/UpToDate
Dostępność usługi	100%

Tabela 6: Wyniki testu 3 - powrót uszkodzonego węzła do klastra

Test został zakończony pomyślnie:

- Usługa HTTP pozostawała dostępna przez cały czas trwania testu.
- DRBD wykonał synchronizację przyrostową jedynie zmienionych bloków danych (1.66 MiB), bez konieczności pełnej synchronizacji dysku.
- Po ponownym uruchomieniu VM2 klastr automatycznie odzyskał stan `Connected, UpToDate/UpToDate`.

3.4 Test 4 - zerwanie VPN

Zrywana jest komunikacja między VM1 i VM2. Celem testu jest sprawdzenie zachowania klastra przy utracie łączności i skomentowanie ryzyka *split-brain*.

W stanie wyjściowym VM1 jest aktywna (Primary), VM2 pasywna (Secondary). Uruchomiono próbnik wysyłający zapytania HTTP `/health` na adres VIP co 0.5 s. Po upływie pięciu sekund na VM2 wykonano polecenie `sudo wg-quick down wg0`, co spowodowało usunięcie interfejsu, na którym kursuje cała komunikacja klastra (Corosync, DRBD oraz adres VIP). W trakcie izolacji, dla weryfikacji że klient pisze do "starego" Primary i dane są bezpieczne, na VM1 wgrano dodatkowy plik testowy `partition-test.bin` o rozmiarze 1 MB. Po czterdziestu sekundach izolacji tunel przywrócono poleceniem `sudo wg-quick up wg0`. Do zarządzania VM2 w trakcie izolacji wykorzystano niezależny tunel Tailscale jako awaryjny kanał administracyjny.

Metryka	Wartość
Liczba żądań HTTP w trakcie testu	120
Żądania udane (HTTP 200)	120 (100%)
Żądania błędne	0
Średnia / maksymalna latencja	1.1 ms / 2 ms
Stan DRBD podczas izolacji (VM1)	WFConnection, Primary/Unknown
Stan DRBD podczas izolacji (VM2)	StandAlone, Primary/Unknown
Promocja DRBD do Primary na obu węzłach	Tak (<i>split-brain</i>)
Aktywacja VIP 10.0.0.100 na VM2	Nie (<i>findif failed</i>)
Plik wgrany na VM1 podczas izolacji	<code>partition-test.bin</code> , 1 MB
Czas izolacji	40 s
Czas od przywrócenia tunelu do Connected	~35 s (po ręcznej interwencji)
Stan DRBD po resynchronizacji	Connected, UpToDate/UpToDate
Wynik <code>drbdadm verify r0</code> (500 MB)	0 sektorów rozbieżnych
Plik z czasu izolacji obecny na VM2 po rekonsyliacji	Tak

Tabela 7: Wyniki testu 4 - zerwanie VPN i resynchronizacja klastra

Test ujawnił następujące zjawiska:

- **Wystąpił *split-brain*** - w czasie izolacji oba węzły lokalnie promowały się do roli DRBD Primary. Polityka `no-quorum-policy=ignore` (wymagana w klastrze dwu-węzłowym) sprawiła, że każdy węzeł uznał się za samotnego ocalałego.
- **Klient nie doświadczył przerwy w usługach** - dostępność 100%. VM1 obsługiwała ruch lokalnie przez cały czas testu, ponieważ to ona posiadała aktywny adres VIP w momencie zerwania tunelu.
- **Naturalna ochrona przed duplikatem VIP** - próba aktywacji adresu 10.0.0.100 na VM2 zakończyła się błędem `findif failed`, ponieważ VIP jest skonfigurowany na interfejsie `wg0`, który właśnie został wyłączony. Klient nigdy nie zobaczyłby dwóch maszyn odpowiadających na ten sam adres.
- **Resynchronizacja DRBD wymagała ręcznej interwencji** - po przywróceniu tunelu oba węzły początkowo pozostały w stanie `StandAlone`. Wymuszenie resynchronizacji nastąpiło poleceniami administracyjnymi `drbdadm disconnect` oraz `drbdadm - -discard-my-data connect r0` po stronie VM2. Polityka `after-sb-1pri discard-secondary` została wówczas zastosowana - VM2 cofnęła swoją lokalną rolę Primary i zaakceptowała stan VM1 jako kanoniczny. Pełna weryfikacja sumami kontrolnymi (`drbdadm verify`) potwierdziła identyczność dysków bit-w-bit, a plik `partition-test.bin` (wgrany wyłącznie na VM1 podczas izolacji) został prawidłowo zreplikowany na VM2.

Ograniczenia. Ochrona przed niespójnością danych wynikała z konkretnej topologii sieci, w której VIP, heartbeat (Corosync) i replikacja (DRBD) korzystają z tego samego interfejsu `wg0`. W typowej konfiguracji produkcyjnej, gdzie VIP siedzi na interfejsie publicznym niezależnym od warstwy klastra, taka ochrona by nie zadziałała. Konieczność ręcznej interwencji w procesie resynchronizacji wskazuje również, że samoczynne wyjście klastra ze stanu *split-brain* w konfiguracji DRBD 8.4 bez fencingu jest zawodne. Właściwe rozwiązania to: **qdevice** (trzeci węzeł arbitra dla naturalnego

kworum), **STONITH** (fizyczny fencing zewnętrzny) lub rozbudowa klastra do co najmniej trzech węzłów.

3.5 Test 5 - awaria podczas uploadu

Rozpoczynany jest upload większego pliku, a w trakcie tej operacji wyłączana jest maszyna **VM1**. Po przełączeniu sprawdzane będzie, czy plik został zapisany poprawnie, częściowo czy wcale.

Wygenerowano plik testowy o rozmiarze 50 MiB wypełniony losowymi bajtami z `/dev/urandom`. Rozmiar dobrano tak, aby upload przez tunel WAN trwał kilka sekund - dając okno czasowe na ingerencję. Upload uruchomiono w tle, a po dwóch sekundach proces aplikacji **fileapp** został zabity sygnałem **SIGKILL** (`kill -9`). Następnie wykonano `pcs node standby vm1`, wymuszając przeniesienie zasobów klastra na **VM2**.

Aplikacja **fileapp** implementuje wzorec atomowego zapisu: dane są najpierw zapisywane do pliku tymczasowego o nazwie `.<nazwa>.<random>.part` w docelowym katalogu, następnie wymuszany jest `fsync()` na pliku, a dopiero potem wykonywany jest `os.rename()` na nazwę docelową (operacja atomowa na poziomie ext4) i `fsync()` na katalogu.

Metryka	Wynik
Rozmiar pliku	52 428 800 B (50 MiB)
Czas do zabicia procesu od startu uploadu	~2 s
Plik tymczasowy <code>.part</code> po crashu	Obecny (52 428 800 B)
Plik docelowy po crashu	Nie istnieje
Reakcja Pacemakera na padnięcie procesu	<code>monitor returned 'Not running'</code>
Stan po failoverze na VM2	Identyczny (<code>.part</code> zreplikowany przez DRBD)
Odpowiedź serwera na <code>GET /files/<nazwa></code>	HTTP 404
Plik dostępny klientowi w jakiegokolwiek formie	Nie
Częściowe / uszkodzone dane widoczne klientowi	Nie

Tabela 8: Wyniki testu 5 - awaria w trakcie uploadu

Plik nie został zapisany **wcale** (z perspektywy klienta), choć jego niekompletna kopia tymczasowa pozostała w filesystemie:

- Aplikacja zdążyła zapisać wszystkie 50 MiB do pliku tymczasowego i wykonać `fsync()`, ale **SIGKILL** trafił przed `os.rename()`. Plik docelowy nigdy nie powstał.
- Pacemaker, niezależnie od wymuszonego standby, samodzielnie wykrył padnięcie procesu aplikacji poprzez okresowy monitoring agenta zasobu i zarejestrował awarię (`'Not running'` (`failed`)). Stanowi to dodatkową demonstrację, że klaster monitoruje stan poszczególnych usług, a nie wyłącznie stan węzła jako całości.
- Po failoverze **VM2** odziedziczyła ten sam stan dyskowy, włącznie z plikiem tymczasowym - DRBD operuje na poziomie urządzenia blokowego i replikuje wszystkie bloki ext4 niezależnie od ich semantyki.

- Klient pytający o oryginalną nazwę pliku otrzymuje czyste HTTP 404, a nie częściowo zapisany plik, który mógłby zostać błędnie zinterpretowany jako poprawny (HTTP nie sprawdza integralności treści ponad `Content-Length`).
- Plik tymczasowy nie jest dostępny przez API - funkcja `_safe_name()` w aplikacji odrzuca żądania o nazwy zaczynające się od kropki (konwencja POSIX dla plików ukrytych).

Wniosek architektoniczny. Skuteczność testu wynika z łańcucha trzech warstw: (1) atomowy zapis w aplikacji, (2) synchroniczna replikacja DRBD (protocol C), (3) failover Pacemakera. Wyłącznie ten zestaw razem gwarantuje, że klient widzi albo pełny niezmieniony plik, albo brak pliku - nigdy stanu pośredniego. Sama replikacja DRBD i sam failover nie wystarczyłyby - gdyby aplikacja zapisywała bezpośrednio pod nazwą docelową, klient po awarii mógłby pobrać częściowy plik, którego korupcja byłaby nieoczywista.